UTS

PEMBELAJARAN MESIN



NAMA : Muhammad Raihan

NIM : 202331108

KELAS : H

DOSEN : Risqia Cahyaningtyas, ST.,M.Kom

NO.PC : 20

ASISTEN : 1. Muhammad Ichwan Fauzi

2. Benidict Samuel Ardy

INSTITUT TEKNOLOGI PLN

TEKNIK INFORMATIKA

2026/2027

DAFTAR ISI

BAB I PENDAHULUAN **3**
1.1 Rumusan Masalah 3
1.2 Tujuan Masalah 3
1.3 Manfaat Masalah 4
BAB II LANDASAN TEORI **5**
2.1 Machine Learning 5
2.2 Klasifikasi 5
2.3 Heart Disease Prediction 5
2.4 Dataset dan Karakteristik Fitur 5
2.5 Exploratory Data Analysis (EDA) 6
2.6 Data Preprocessing 6
2.7 Encoding Data Kategorikal 6
2.8 Standardisasi Fitur 7
2.9 Pembagian Data Training dan Testing 7
2.10 Algoritma Decision Tree 7
2.11 Algoritma K-Nearest Neighbors (KNN) 7
2.12 Confusion Matrix 8
2.13 Accuracy, Precision, Recall, dan F1-score 8
2.14 Evaluasi dan Perbandingan Model Klasifikasi 8
BAB III HASIL **9**
3.1 Persiapan Lingkungan dan Pemuatan Data 9
3.2 Data Understanding 10
3.3 Exploratory Data Analysis (EDA) 12
3.4 Data Preprocessing dan Encoding 19
3.5 Model dan Evaluasi Decision Tree 21
3.6 Model dan Evaluasi KNN 25
3.7 Perbandingan Decision Tree dan KNN 27
BAB IV PENUTUP **31**
DAFTAR PUSTAKA **32**

BAB I
PENDAHULUAN

1.1 Rumusan Masalah

Penyakit jantung merupakan salah satu masalah kesehatan yang membutuhkan deteksi dini. Proses diagnosis melibatkan banyak parameter klinis sekaligus, misalnya usia, jenis nyeri dada, tekanan darah, kolesterol, hasil elektrokardiogram (EKG), detak jantung maksimum, dan respons tubuh saat berolahraga. Pendekatan Machine Learning memungkinkan pola dari parameter-parameter tersebut dipelajari secara otomatis untuk memprediksi apakah seorang pasien berisiko penyakit jantung (Qumsiyeh et al., 2026).

Dataset yang digunakan pada UTS ini adalah 01_heart_disease.csv yang berisi 918 pasien dengan 11 fitur dan satu target biner HeartDisease (0 = normal, 1 = penyakit jantung). Permasalahannya adalah masalah klasifikasi biner: bagaimana membangun model yang dapat memprediksi kelas HeartDisease dari fitur-fitur tersebut dengan akurat. Dataset ini juga memiliki tantangan data, yaitu nilai nol yang tidak wajar pada Cholesterol dan RestingBP serta kombinasi fitur numerik dan kategorikal yang harus ditangani sebelum pemodelan.

Dua algoritma yang ditetapkan untuk tugas ini adalah Decision Tree dan K-Nearest Neighbors (KNN). Rumusan masalah yang dijawab adalah: (1) bagaimana karakteristik dataset dan hubungan antar fitur dengan target berdasarkan EDA; (2) bagaimana preprocessing yang tepat tanpa data leakage; (3) bagaimana performa Decision Tree dan KNN dalam memprediksi penyakit jantung; dan (4) algoritma mana yang lebih sesuai untuk dataset ini berdasarkan hasil eksperimen.

1.2 Tujuan Masalah

Tujuan praktikum UTS ini adalah sebagai berikut.

1. Memahami struktur dataset melalui EDA dan menyiapkan data melalui preprocessing (penanganan nilai tidak wajar, encoding, standardisasi, dan pembagian data 80:20 secara stratified) tanpa data leakage.

2. Menerapkan algoritma Decision Tree untuk memprediksi penyakit jantung, menentukan hiperparameter yang sesuai, dan mengevaluasinya menggunakan accuracy, precision, recall, F1-score, dan confusion matrix.

3. Menerapkan algoritma KNN pada data yang sudah distandardisasi, menguji nilai K 1 sampai 30 untuk memperoleh K terbaik, lalu mengevaluasinya dengan metrik yang sama.

4. Membandingkan performa Decision Tree dan KNN berdasarkan angka hasil eksekusi program untuk menentukan algoritma yang lebih sesuai pada dataset ini.

1.3 Manfaat Masalah

Manfaat yang diperoleh dari praktikum UTS ini adalah sebagai berikut.

1. Memahami alur Machine Learning secara utuh, mulai dari data understanding, EDA, preprocessing, pelatihan model, hingga evaluasi, serta pentingnya menjaga data testing agar tidak bocor ke proses pelatihan.

2. Memahami karakteristik data kesehatan, khususnya data klinis yang memuat nilai tidak wajar dan fitur campuran (numerik dan kategorikal), serta cara menanganinya.

3. Mengetahui kelebihan dan kekurangan Decision Tree dan KNN pada kasus nyata berdasarkan angka hasil eksperimen, sehingga pemilihan algoritma dapat didasarkan pada bukti dan bukan asumsi.

BAB II
LANDASAN TEORI

Landasan teori berikut disusun dari jurnal ilmiah terbitan 2021–2026 dan dihubungkan langsung dengan praktik yang dikerjakan pada BAB III.

2.1 Machine Learning

Machine Learning (ML) adalah cabang kecerdasan buatan yang membuat sistem mampu belajar pola dari data tanpa diprogram dengan aturan eksplisit. Sarker (2021) mengelompokkan ML menjadi pembelajaran terawasi (supervised), tidak terawasi (unsupervised), semi-terawasi, dan reinforcement learning, serta menekankan bahwa pemilihan algoritma bergantung pada jenis data dan tujuan masalah. Pada praktikum ini digunakan pembelajaran terawasi karena data memiliki label target HeartDisease yang sudah diketahui.

2.2 Klasifikasi

Klasifikasi adalah tugas supervised learning yang memetakan data ke dalam kelas diskret berdasarkan fitur masukan (Sarker, 2021). Jika hanya ada dua kelas, tugas ini disebut klasifikasi biner. Kualitas model klasifikasi dinilai dari kesesuaian antara label prediksi dan label sebenarnya pada data yang belum pernah dilihat model (Tharwat, 2021). Pada praktikum ini, klasifikasi biner dipakai untuk membedakan pasien normal (0) dan pasien penyakit jantung (1).

2.3 Heart Disease Prediction

Prediksi penyakit jantung dengan ML memanfaatkan data klinis rutin untuk mengestimasi risiko pasien. Qumsiyeh et al. (2026) menggunakan dataset 918 pasien dengan 11 prediktor klinis untuk memprediksi penyakit jantung, membandingkan Logistic Regression, Random Forest, KNN, dan Decision Tree, serta melaporkan bahwa pada data uji mereka KNN memberikan akurasi 0,917 sedangkan Decision Tree 0,851. Penelitian lain memperlihatkan hasil yang berbeda: Budi (2026) pada dataset 205 data melaporkan Decision Tree (98,54%) sedikit melampaui KNN (98,05%), dan Pavithraa dan Sivaprasad (2022) melaporkan Decision Tree (84,95%) lebih baik daripada KNN (76,29%). Perbedaan ini menunjukkan bahwa performa algoritma sangat dipengaruhi oleh dataset, ukuran sampel, dan skema preprocessing, sehingga perbandingan harus dilakukan secara empiris pada data yang dipakai.

2.4 Dataset dan Karakteristik Fitur

Dataset 01_heart_disease.csv memiliki 918 baris dan 12 kolom, identik dengan jumlah observasi dan prediktor pada dataset yang dipakai Qumsiyeh et al. (2026). Fitur numerik terdiri atas Age, RestingBP (tekanan darah istirahat), Cholesterol, MaxHR (detak jantung maksimum), dan Oldpeak

(depresi segmen ST akibat olahraga). Fitur kategorikal terdiri atas Sex, ChestPainType (TA, ATA, NAP, ASY), RestingECG (Normal, ST, LVH), ExerciseAngina (Y/N), dan ST_Slope (Up, Flat, Down). FastingBS adalah fitur biner 0/1 yang menandai gula darah puasa di atas ambang tertentu, dan HeartDisease adalah target.

Karakteristik penting dataset ini adalah adanya nilai nol yang tidak mungkin secara klinis pada RestingBP dan Cholesterol. Qumsiyeh et al. (2026) memperlakukan nilai tidak wajar tersebut sebagai data hilang lalu mengimputasinya dengan median; pendekatan yang sama diterapkan pada praktikum ini.

2.5 Exploratory Data Analysis (EDA)

EDA adalah tahap pemeriksaan awal data untuk memahami struktur, distribusi, kualitas, dan hubungan antar variabel sebelum pemodelan. Dalam alur kerja ML, pemahaman data merupakan bagian dari pipeline yang menentukan keputusan preprocessing selanjutnya (Sarker, 2021). Pada praktikum ini, EDA meliputi pemeriksaan ukuran data, tipe data, statistik deskriptif, missing value, duplikat, distribusi fitur dan target, korelasi, serta outlier. Temuan EDA, misalnya kolesterol bernilai nol, menjadi dasar langkah preprocessing (Qumsiyeh et al., 2026).

2.6 Data Preprocessing

Data preprocessing adalah rangkaian transformasi untuk mengubah data mentah menjadi bentuk yang layak dipelajari algoritma, meliputi pembersihan data, penanganan nilai hilang dan outlier, encoding, dan penskalaan (Sarker, 2021). Qumsiyeh et al. (2026) menerapkan imputasi median untuk nilai numerik tidak wajar, one-hot encoding untuk fitur kategorikal, dan standardisasi z-score untuk fitur numerik, dengan seluruh langkah dipelajari hanya dari data latih di dalam pipeline validasi silang. Praktikum ini mengikuti prinsip tersebut dengan Pipeline dan ColumnTransformer dari Scikit-Learn.

2.7 Encoding Data Kategorikal

Algoritma ML pada umumnya memerlukan masukan numerik, sehingga fitur kategorikal harus dikodekan. One-hot encoding mengubah satu fitur kategorikal dengan m kategori menjadi m kolom biner sehingga tidak ada urutan semu antar kategori; metode ini dipakai oleh Qumsiyeh et al. (2026) pada dataset penyakit jantung yang sama. Pada praktikum ini one-hot encoding dipilih karena ChestPainType, RestingECG, dan ST_Slope bersifat nominal. Fitur biner (Sex, ExerciseAngina) cukup direpresentasikan satu kolom (drop='if_binary'), sehingga total fitur setelah encoding menjadi 18.

2.8 Standardisasi Fitur

Standardisasi z-score mengubah fitur menjadi rata-rata 0 dan simpangan baku 1 dengan rumus $z = (x - \mu) / \sigma$. Penskalaan penting bagi algoritma berbasis jarak seperti KNN karena fitur dengan rentang besar (misalnya Cholesterol) akan mendominasi jarak. Pratama dan Salam (2025) membandingkan teknik normalisasi pada KNN dan melaporkan bahwa penskalaan meningkatkan kinerja KNN secara signifikan dibandingkan data tanpa normalisasi. Decision Tree memilih pemisah berdasarkan ambang pada satu fitur sehingga tidak bergantung pada skala; karena itu standardisasi hanya diterapkan pada jalur KNN.

2.9 Pembagian Data Training dan Testing

Data dibagi menjadi data latih untuk membangun model dan data uji untuk mengukur performa pada data baru. Pembagian 80:20 digunakan pada praktikum ini, sejalan dengan skema 80:20 yang dipakai Budi (2026). Parameter stratify=y menjaga proporsi kelas target tetap sama pada kedua subset. Kapoor dan Narayanan (2023) menunjukkan bahwa kebocoran data (data leakage), yaitu informasi dari data uji yang masuk ke proses pelatihan, menyebabkan estimasi performa terlalu optimistis dan menimbulkan masalah reproduktibilitas pada penelitian berbasis ML. Untuk mencegahnya, split dilakukan lebih dulu dan seluruh imputasi, scaling, serta pemilihan hiperparameter hanya memakai data latih.

2.10 Algoritma Decision Tree

Decision Tree adalah model berbentuk pohon yang membagi data secara rekursif berdasarkan pengujian pada fitur hingga tiap simpul daun mewakili satu kelas (Sarker, 2021). Pemisahan dipilih untuk menurunkan impurity, misalnya $\text{Gini} = 1 - \Sigma\, p_i^2$ atau $\text{Entropy} = -\Sigma\, p_i \log_2 p_i$, dengan p_i proporsi kelas i pada simpul. Keunggulan utamanya adalah interpretabilitas karena keputusan dapat dibaca sebagai aturan jika-maka, sedangkan kelemahannya adalah kecenderungan overfitting bila pohon dibiarkan tumbuh dalam. Karena itu hiperparameter seperti max_depth dan min_samples_leaf dibatasi. Decision Tree telah dipakai untuk prediksi penyakit jantung dan memberi hasil yang kompetitif pada beberapa studi (Budi, 2026; Pavithraa dan Sivaprasad, 2022).

2.11 Algoritma K-Nearest Neighbors (KNN)

KNN adalah algoritma berbasis instance yang memprediksi kelas data baru berdasarkan mayoritas kelas dari K tetangga terdekat pada data latih (Sarker, 2021). Kedekatan umumnya diukur dengan jarak Euclidean, $d(x, y) = \sqrt{(\Sigma\, (x_j - y_j)^2)}$. Nilai K yang terlalu kecil membuat model sensitif terhadap noise (overfitting), sedangkan K yang terlalu besar membuat batas keputusan terlalu halus. Karena bergantung pada jarak, KNN membutuhkan fitur yang sudah diskalakan (Pratama dan Salam, 2025). Pada studi dataset penyakit jantung berukuran 918, KNN yang dikombinasikan dengan

preprocessing yang tepat memberi performa terbaik di antara empat model yang dibandingkan (Qumsiyeh et al., 2026).

2.12 Confusion Matrix

Confusion matrix adalah tabel yang merangkum hasil prediksi klasifikasi terhadap label sebenarnya. Untuk klasifikasi biner, matriks ini berisi True Positive (TP), True Negative (TN), False Positive (FP), dan False Negative (FN), dan seluruh metrik evaluasi turunan dihitung dari keempat nilai tersebut (Tharwat, 2021). Pada konteks medis, FN berarti pasien sakit yang diprediksi normal, kesalahan yang umumnya lebih berisiko daripada FP.

2.13 Accuracy, Precision, Recall, dan F1-score

Menurut Tharwat (2021), Accuracy = (TP + TN) / (TP + TN + FP + FN) menyatakan proporsi prediksi benar; Precision = TP / (TP + FP) menyatakan ketepatan prediksi kelas positif; Recall = TP / (TP + FN) menyatakan kemampuan menemukan seluruh kasus positif; dan F1-score = 2 × Precision × Recall / (Precision + Recall) adalah rata-rata harmonik precision dan recall. Accuracy dapat menyesatkan pada data tidak seimbang, sehingga metrik lain perlu dilaporkan bersama. Canbek et al. (2022) juga menunjukkan bahwa penamaan dan pelaporan metrik kerap tidak konsisten antar studi, sehingga definisi metrik pada laporan ini dinyatakan secara eksplisit dengan kelas positif = penyakit jantung (1).

2.14 Evaluasi dan Perbandingan Model Klasifikasi

Perbandingan model harus memakai skema evaluasi yang sama dan data uji yang tidak dipakai untuk memilih hiperparameter (Kapoor dan Narayanan, 2023). Selain satu kali pembagian data uji, validasi silang (cross-validation) memberi estimasi yang lebih stabil, dan Qumsiyeh et al. (2026) menggunakan grid search dengan stratified cross-validation untuk menyetel model. Praktikum ini memakai GridSearchCV 5-fold (Decision Tree) dan cross-validation 5-fold untuk memilih K (KNN) pada data latih, lalu mengevaluasi satu kali pada data uji, dan sebagai uji ketahanan menambahkan Repeated Stratified 5-Fold CV. ROC-AUC dilaporkan sebagai pelengkap karena mengukur kemampuan pemisahan kelas pada seluruh ambang probabilitas (Tharwat, 2021).

BAB III

HASIL

Bab ini menjelaskan seluruh proses praktikum dari awal sampai akhir. Setiap cell kode ditampilkan lengkap dengan output hasil eksekusi nyata dari notebook NIM_UTS_PM.ipynb, kemudian dijelaskan tujuan, proses, dan hasil yang diperoleh. Seluruh sel kode diberi watermark komentar NIM pada baris pertama.

3.1 Persiapan Lingkungan dan Pemuatan Data

Cell 1

```
# Watermark NIM: 202331108
import os, json, warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
warnings.filterwarnings('ignore')

from sklearn.model_selection import train_test_split, GridSearchCV, StratifiedKFold, RepeatedStratifiedKFold, cross_val_score
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.tree import DecisionTreeClassifier, plot_tree, export_text
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import (accuracy_score, precision_score, recall_score, f1_score, roc_auc_score,
                             confusion_matrix, ConfusionMatrixDisplay, classification_report, roc_curve)

RANDOM_STATE = 42
os.makedirs('fig', exist_ok=True)
sns.set_theme(style='whitegrid')
plt.rcParams['figure.dpi'] = 110

HASIL = {}   # menampung seluruh angka hasil eksperimen

def simpan(nama):
    plt.tight_layout()
    plt.savefig(f'fig/{nama}.png', dpi=150, bbox_inches='tight')
    plt.show()

print('Library siap. random_state =', RANDOM_STATE)
```

Output:

```
Library siap. random_state = 42
```

Penjelasan

Cell ini menyiapkan seluruh library: pandas dan numpy untuk data, matplotlib dan seaborn untuk grafik, serta scikit-learn untuk preprocessing, model, dan evaluasi. Pipeline dan ColumnTransformer sengaja dipakai supaya imputasi, encoding, dan scaling selalu di-fit hanya pada data training sehingga tidak terjadi data leakage. RANDOM_STATE = 42 dipakai di seluruh proses agar hasil sama setiap kode dijalankan ulang, dan fungsi simpan() menyimpan setiap grafik ke folder fig. Outputnya hanya pesan konfirmasi bahwa library siap.

Cell 2

```
# Watermark NIM: 202331108
FILE = '01_heart_disease.csv'
if not os.path.exists(FILE):
    try:
        from google.colab import files
        files.upload()
    except ImportError:
        raise FileNotFoundError(f'{FILE} tidak ditemukan. Letakkan di folder yang sama dengan notebook.')
```

```
df = pd.read_csv(FILE)
print('Jumlah baris  :', df.shape[0])
print('Jumlah kolom  :', df.shape[1])
```

Output:

```
Jumlah baris  : 918
Jumlah kolom  : 12
```

Penjelasan

Dataset dibaca dengan pd.read_csv(). Ada pengecekan file agar notebook tetap bisa dijalankan di Google Colab (file diminta diunggah jika belum ada). Hasilnya, dataset memiliki **918 baris dan 12 kolom** (11 fitur dan 1 target).

3.2 Data Understanding

Cell 3

```
# Watermark NIM: 202331108
print('=== 5 DATA PERTAMA ===')
display(df.head())
print('=== 5 DATA TERAKHIR ===')
display(df.tail())
```

Output:

```
=== 5 DATA PERTAMA ===

   Age Sex ChestPainType  RestingBP  Cholesterol  FastingBS RestingECG  MaxHR  \
0   40   M           ATA        140          289          0     Normal    172
1   49   F           NAP        160          180          0     Normal    156
2   37   M           ATA        130          283          0         ST     98
3   48   F           ASY        138          214          0     Normal    108
4   54   M           NAP        150          195          0     Normal    122

  ExerciseAngina  Oldpeak ST_Slope  HeartDisease
0              N      0.0       Up             0
1              N      1.0     Flat             1
2              N      0.0       Up             0
3              Y      1.5     Flat             1
4              N      0.0       Up             0
=== 5 DATA TERAKHIR ===

     Age Sex ChestPainType  RestingBP  Cholesterol  FastingBS RestingECG  \
913   45   M           TA        110          264          0     Normal
914   68   M          ASY        144          193          1     Normal
915   57   M          ASY        130          131          0     Normal
916   57   F          ATA        130          236          0        LVH
917   38   M          NAP        138          175          0     Normal

     MaxHR ExerciseAngina  Oldpeak ST_Slope  HeartDisease
913    132              N      1.2     Flat             1
914    141              N      3.4     Flat             1
915    115              Y      1.2     Flat             1
916    174              N      0.0     Flat             1
917    173              N      0.0       Up             0
```

Penjelasan

Cell ini menampilkan lima data pertama dan terakhir dengan head() dan tail() untuk melihat bentuk data secara langsung. Terlihat bahwa dataset memuat fitur numerik (Age, RestingBP, Cholesterol, MaxHR, Oldpeak) dan kategorikal berupa teks (Sex, ChestPainType, RestingECG, ExerciseAngina, ST_Slope), sedangkan HeartDisease sudah berupa angka 0/1. Karena ada fitur teks, encoding akan diperlukan.

Cell 4

```
# Watermark NIM: 202331108
print('=== INFO / TIPE DATA ===')
df.info()
```

Output:

```
=== INFO / TIPE DATA ===
<class 'pandas.DataFrame'>
RangeIndex: 918 entries, 0 to 917
Data columns (total 12 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   Age             918 non-null    int64
 1   Sex             918 non-null    str
 2   ChestPainType   918 non-null    str
 3   RestingBP       918 non-null    int64
 4   Cholesterol     918 non-null    int64
 5   FastingBS       918 non-null    int64
 6   RestingECG      918 non-null    str
 7   MaxHR           918 non-null    int64
 8   ExerciseAngina  918 non-null    str
 9   Oldpeak         918 non-null    float64
 10  ST_Slope        918 non-null    str
 11  HeartDisease    918 non-null    int64
dtypes: float64(1), int64(6), str(5)
memory usage: 86.2 KB
```

Penjelasan

df.info() menampilkan tipe data tiap kolom. Ada 12 kolom dengan 918 nilai non-null pada semuanya; enam kolom bertipe int64 (Age, RestingBP, Cholesterol, FastingBS, MaxHR, HeartDisease), satu float64 (Oldpeak), dan lima kolom teks. FastingBS bertipe angka tetapi sebenarnya fitur biner 0/1, sehingga diperlakukan khusus saat preprocessing.

Cell 5

```
# Watermark NIM: 202331108
print('=== STATISTIK DESKRIPTIF (numerik) ===')
display(df.describe().T.round(2))
print('=== STATISTIK DESKRIPTIF (kategorikal) ===')
display(df.describe(include='object').T)
```

Output:

```
=== STATISTIK DESKRIPTIF (numerik) ===

             count    mean     std  min     25%    50%    75%    max
Age          918.0   53.51    9.43  28.0   47.00   54.0   60.0   77.0
RestingBP    918.0  132.40   18.51   0.0  120.00  130.0  140.0  200.0
Cholesterol  918.0  198.80  109.38   0.0  173.25  223.0  267.0  603.0
FastingBS    918.0    0.23    0.42   0.0    0.00    0.0    0.0    1.0
MaxHR        918.0  136.81   25.46  60.0  120.00  138.0  156.0  202.0
Oldpeak      918.0    0.89    1.07  -2.6    0.00    0.6    1.5    6.2
HeartDisease 918.0    0.55    0.50   0.0    0.00    1.0    1.0    1.0
=== STATISTIK DESKRIPTIF (kategorikal) ===

              count unique     top freq
Sex             918      2       M  725
ChestPainType   918      4     ASY  496
RestingECG      918      3  Normal  552
ExerciseAngina  918      2       N  547
ST_Slope        918      3    Flat  460
```

Penjelasan

describe() memberi statistik deskriptif. Usia rata-rata 53,51 tahun (rentang 28 sampai 77). Ada dua temuan penting: nilai minimum RestingBP adalah 0 dan minimum Cholesterol adalah 0, yang tidak mungkin secara medis. Rata-rata Cholesterol (198,80) juga lebih rendah daripada mediannya (223), tanda bahwa nilai nol menarik rata-rata ke bawah. Oldpeak memiliki nilai minimum −2,6. Pada fitur kategorikal, pasien laki-laki mendominasi (725 dari 918 atau sekitar 78,98%) dan tipe nyeri dada terbanyak adalah ASY (496 pasien).

3.3 Exploratory Data Analysis (EDA)

Cell 6

```
# Watermark NIM: 202331108
print('Missing value (NaN) per kolom:')
print(df.isnull().sum())
print('\nJumlah baris duplikat:', df.duplicated().sum())

# Pemeriksaan nilai tidak wajar secara medis
print('\nRestingBP == 0   :', (df['RestingBP'] == 0).sum(), 'baris')
print('Cholesterol == 0 :', (df['Cholesterol'] == 0).sum(), 'baris')
print('Oldpeak < 0      :', (df['Oldpeak'] < 0).sum(), 'baris')
HASIL['eda'] = {'baris': int(df.shape[0]), 'kolom': int(df.shape[1]), 'missing_nan': int(df.isnull().sum().sum()),
                'duplikat': int(df.duplicated().sum()), 'restingbp_nol': int((df['RestingBP']==0).sum()),
                'cholesterol_nol': int((df['Cholesterol']==0).sum()), 'oldpeak_negatif': int((df['Oldpeak']<0).sum())}
```

Output:

```
Missing value (NaN) per kolom:
Age             0
Sex             0
ChestPainType   0
RestingBP       0
Cholesterol     0
FastingBS       0
RestingECG      0
MaxHR           0
ExerciseAngina  0
Oldpeak         0
ST_Slope        0
HeartDisease    0
dtype: int64

Jumlah baris duplikat: 0

RestingBP == 0   : 1 baris
Cholesterol == 0 : 172 baris
Oldpeak < 0      : 13 baris
```

Penjelasan

Cell ini memeriksa kualitas data. Tidak ada missing value (NaN) dan tidak ada baris duplikat, sehingga tidak ada baris yang perlu dihapus. Namun pengecekan nilai tidak wajar menemukan **missing value tersembunyi**: Cholesterol = 0 pada 172 baris (sekitar 18,74% data), RestingBP = 0 pada 1 baris, dan Oldpeak negatif pada 13 baris. Nilai nol pada kolesterol dan tekanan darah akan ditangani saat preprocessing, sedangkan Oldpeak negatif masih mungkin secara klinis sehingga dipertahankan.

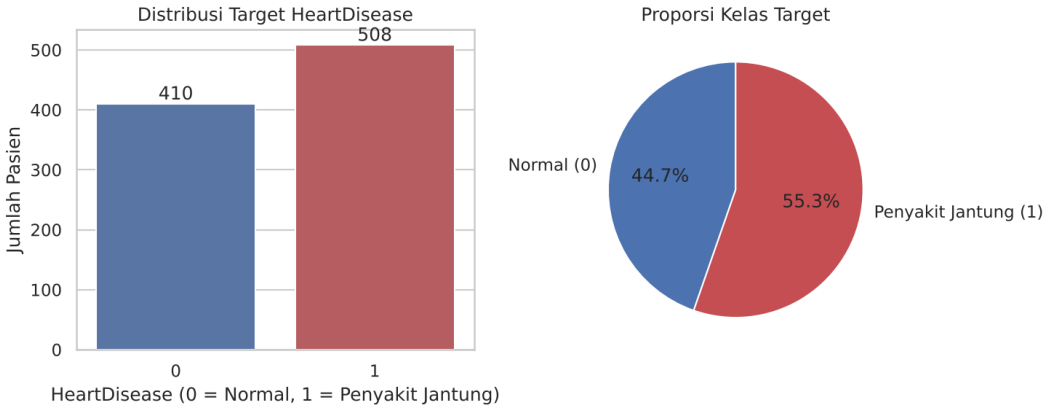
Cell 7

```
# Watermark NIM: 202331108
vc = df['HeartDisease'].value_counts().sort_index()
pct = (df['HeartDisease'].value_counts(normalize=True).sort_index()*100).round(2)
print(pd.DataFrame({'Jumlah': vc, 'Persen (%)': pct}))
HASIL['target'] = {'kelas0': int(vc[0]), 'kelas1': int(vc[1]), 'pct0': float(pct[0]), 'pct1': float(pct[1])}

fig, ax = plt.subplots(1, 2, figsize=(10, 4))
sns.countplot(x='HeartDisease', data=df, ax=ax[0], palette=['#4C72B0', '#C44E52'])
for p in ax[0].patches:
    ax[0].annotate(int(p.get_height()), (p.get_x()+p.get_width()/2, p.get_height()), ha='center', va='bottom')
ax[0].set_title('Distribusi Target HeartDisease')
ax[0].set_xlabel('HeartDisease (0 = Normal, 1 = Penyakit Jantung)'); ax[0].set_ylabel('Jumlah Pasien')
ax[1].pie(vc, labels=['Normal (0)', 'Penyakit Jantung (1)'], autopct='%1.1f%%', colors=['#4C72B0', '#C44E52'], startangle=90)
ax[1].set_title('Proporsi Kelas Target')
simpan('01_target')
```

Output:

```
              Jumlah  Persen (%)
HeartDisease
0                410       44.66
1                508       55.34
```



Gambar: Distribusi target HeartDisease

Penjelasan

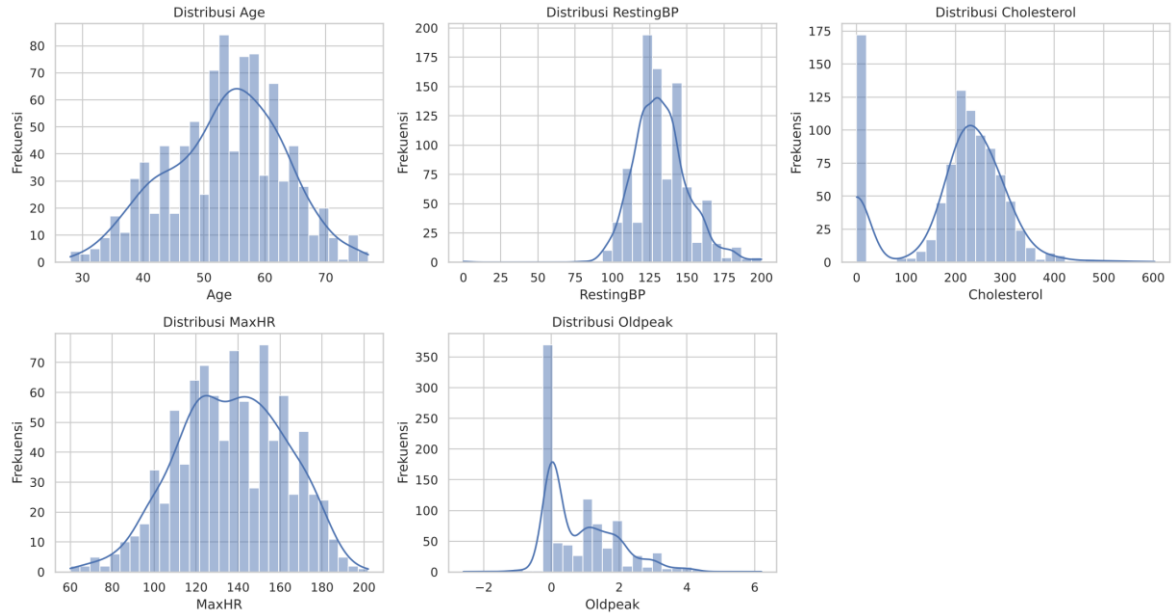
Distribusi target menunjukkan 410 pasien normal (44,66%) dan 508 pasien penyakit jantung (55,34%). Kelas relatif seimbang sehingga accuracy masih layak dipakai, tetapi precision, recall, dan F1-score tetap dilaporkan. Split data nanti memakai stratify=y agar proporsi ini terjaga.

Cell 8

```
# Watermark NIM: 202331108
num_cols_eda = ['Age', 'RestingBP', 'Cholesterol', 'MaxHR', 'Oldpeak']
fig, axes = plt.subplots(2, 3, figsize=(15, 8))
for ax, c in zip(axes.ravel(), num_cols_eda):
    sns.histplot(df[c], kde=True, ax=ax, color='#4C72B0', bins=30)
    ax.set_title(f'Distribusi {c}'); ax.set_xlabel(c); ax.set_ylabel('Frekuensi')
axes[1, 2].axis('off')
simpan('02_hist_numerik')
print('Skewness:'); print(df[num_cols_eda].skew().round(3))
```

Output:

```
Skewness:
Age          -0.196
RestingBP     0.180
Cholesterol  -0.610
MaxHR        -0.144
Oldpeak       1.023
dtype: float64
```



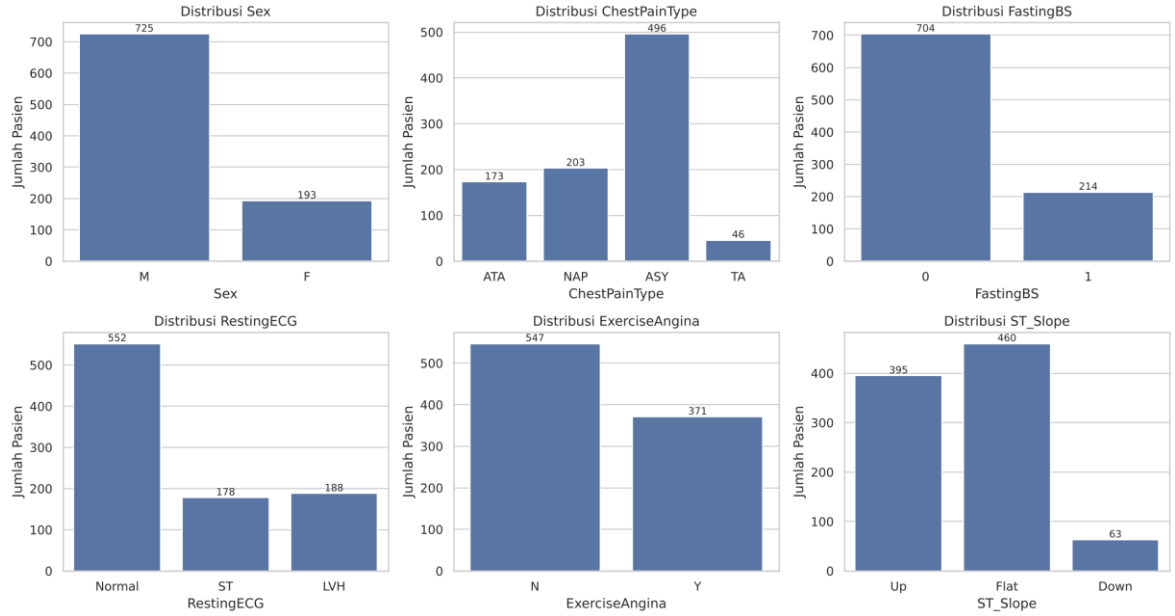
Gambar: Histogram fitur numerik

Penjelasan

Histogram dengan kurva KDE menunjukkan bentuk distribusi fitur numerik. Age dan MaxHR mendekati simetris (skewness −0,196 dan −0,144), RestingBP mendekati simetris (0,180), Cholesterol miring ke kiri (−0,610) dengan lonjakan di nilai 0 akibat data tidak wajar, dan Oldpeak miring ke kanan (1,023) karena banyak pasien bernilai 0 dengan sebagian kecil bernilai tinggi.

Cell 9

```
# Watermark NIM: 202331108
cat_cols_eda = ['Sex', 'ChestPainType', 'FastingBS', 'RestingECG', 'ExerciseAngina', 'ST_Slope']
fig, axes = plt.subplots(2, 3, figsize=(15, 8))
for ax, c in zip(axes.ravel(), cat_cols_eda):
    sns.countplot(x=c, data=df, ax=ax, color='#4C72B0')
    ax.set_title(f'Distribusi {c}'); ax.set_xlabel(c); ax.set_ylabel('Jumlah Pasien')
    for p in ax.patches:
        ax.annotate(int(p.get_height()), (p.get_x()+p.get_width()/2, p.get_height()), ha='center', va='bottom', fontsize=9)
simpan('03_count_kategorikal')
```



Gambar: Countplot fitur kategorikal

Penjelasan

Countplot menunjukkan sebaran fitur kategorikal. Pasien laki-laki jauh lebih banyak daripada perempuan, nyeri dada tipe ASY paling dominan, mayoritas pasien memiliki FastingBS = 0, hasil EKG istirahat paling sering Normal, angina akibat olahraga lebih banyak 'N', dan ST_Slope terbanyak adalah Flat. Hal ini penting karena ketidakseimbangan kategori dapat memengaruhi model.

Cell 10

```
# Watermark NIM: 202331108
fig, axes = plt.subplots(2, 3, figsize=(15, 8))
for ax, c in zip(axes.ravel(), cat_cols_eda):
    sns.countplot(x=c, hue='HeartDisease', data=df, ax=ax, palette=['#4C72B0', '#C44E52'])
    ax.set_title(f'{c} terhadap HeartDisease'); ax.set_xlabel(c); ax.set_ylabel('Jumlah Pasien')
    ax.legend(title='HeartDisease')
simpan('04_kategorikal_vs_target')

print('Persentase pasien berpenyakit jantung per kategori (%):')
rate = {}
for c in cat_cols_eda:
    r = (df.groupby(c)['HeartDisease'].mean()*100).round(1)
    rate[c] = r.to_dict(); print(f'\n{c}\n{r.rename_axis(None).to_string()}')
HASIL['rate_kategori'] = {k: {str(a): b for a, b in v.items()} for k, v in rate.items()}
```

Output:

```
Persentase pasien berpenyakit jantung per kategori (%):

Sex
F    25.9
M    63.2

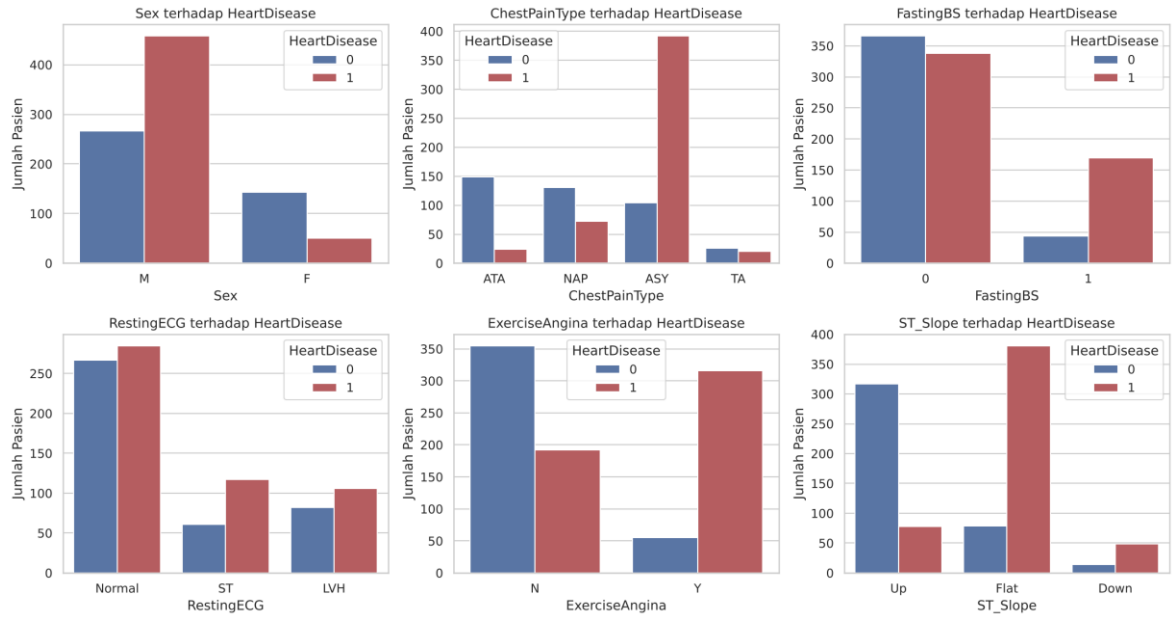
ChestPainType
ASY    79.0
ATA    13.9
NAP    35.5
TA     43.5

FastingBS
0    48.0
1    79.4

RestingECG
LVH       56.4
Normal    51.6
ST        65.7

ExerciseAngina
N    35.1
Y    85.2

ST_Slope
Down    77.8
Flat    82.8
Up      19.7
```



Gambar: Fitur kategorikal terhadap target

Penjelasan

Grafik ini membandingkan tiap fitur kategorikal dengan target, ditambah persentase pasien berpenyakit jantung per kategori. Hubungan paling kuat: ExerciseAngina = Y (85,2% sakit vs 35,1% untuk N), ST_Slope Flat (82,8%) dan Down (77,8%) vs Up (19,7%), ChestPainType ASY (79,0%) vs ATA (13,9%), dan Sex laki-laki (63,2%) vs perempuan (25,9%). FastingBS = 1 memiliki persentase sakit 79,4% (vs 48,0%), sedangkan RestingECG hubungannya lebih lemah (Normal 51,6%, LVH 56,4%, ST 65,7%).

Cell 11

```
# Watermark NIM: 202331108
fig, axes = plt.subplots(2, 3, figsize=(15, 8))
for ax, c in zip(axes.ravel(), num_cols_eda):
```

```
    sns.boxplot(x='HeartDisease', y=c, data=df, ax=ax, palette=['#4C72B0', '#C44E52'])
    ax.set_title(f'{c} berdasarkan HeartDisease'); ax.set_xlabel('HeartDisease'); ax.set_ylabel(c)
axes[1, 2].axis('off')
simpan('05_boxplot_numerik_vs_target')
print('Rata-rata fitur numerik per kelas target:')
display(df.groupby('HeartDisease')[num_cols_eda].mean().round(2))

# Cek pengaruh nilai Cholesterol = 0 terhadap rata-rata
print('\nJumlah pasien dengan Cholesterol = 0 per kelas target:')
print(pd.crosstab(df['Cholesterol'] == 0, df['HeartDisease'], rownames=['Cholesterol == 0'], colnames=['HeartDisease']))
print('\nRata-rata Cholesterol per kelas (tanpa nilai 0):')
print(df[df['Cholesterol'] > 0].groupby('HeartDisease')['Cholesterol'].mean().round(2))
```

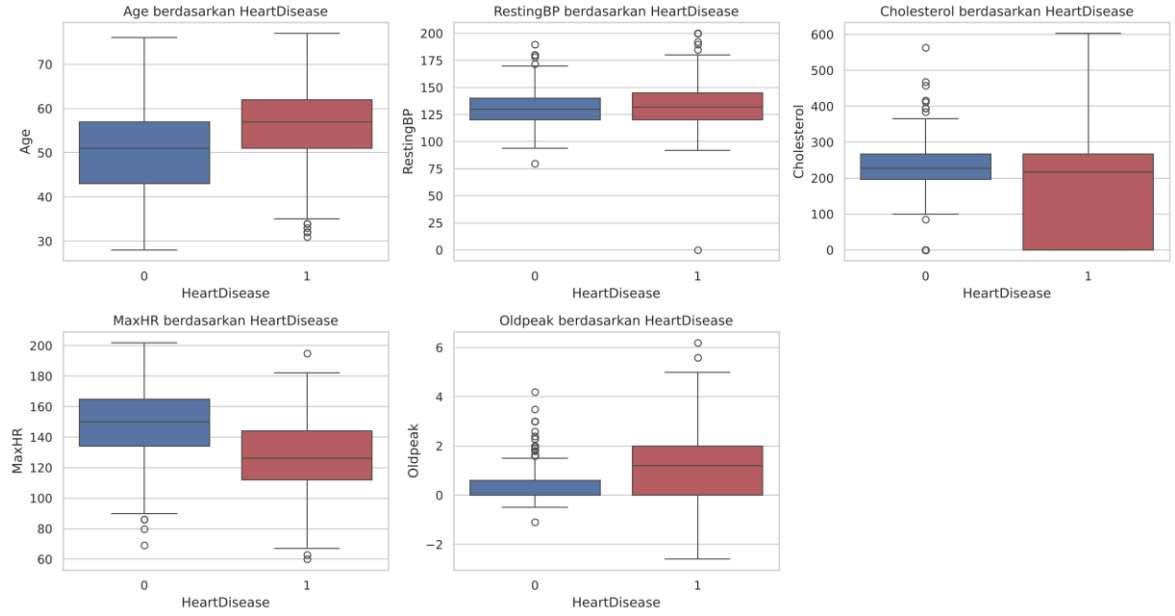
Output:

```
Rata-rata fitur numerik per kelas target:

              Age  RestingBP  Cholesterol   MaxHR  Oldpeak
HeartDisease
0           50.55     130.18       227.12  148.15     0.41
1           55.90     134.19       175.94  127.66     1.27

Jumlah pasien dengan Cholesterol = 0 per kelas target:
HeartDisease       0    1
Cholesterol == 0
False            390  356
True              20  152

Rata-rata Cholesterol per kelas (tanpa nilai 0):
HeartDisease
0    238.77
1    251.06
Name: Cholesterol, dtype: float64
```



Gambar: Boxplot fitur numerik terhadap target

Penjelasan

Boxplot dan rata-rata per kelas memperlihatkan perbedaan fitur numerik. Pasien berpenyakit jantung rata-rata lebih tua (55,90 vs 50,55 tahun), memiliki MaxHR lebih rendah (127,66 vs 148,15), dan Oldpeak lebih tinggi (1,27 vs 0,41). Rata-rata Cholesterol pasien sakit tampak lebih rendah (175,94 vs 227,12), namun ini artefak: dari 172 baris kolesterol nol, 152 berasal dari pasien sakit. Setelah nilai nol dikeluarkan, rata-rata kolesterol pasien sakit justru sedikit lebih tinggi (251,06 vs 238,77). Temuan ini mendukung keputusan memperlakukan nol sebagai missing value.

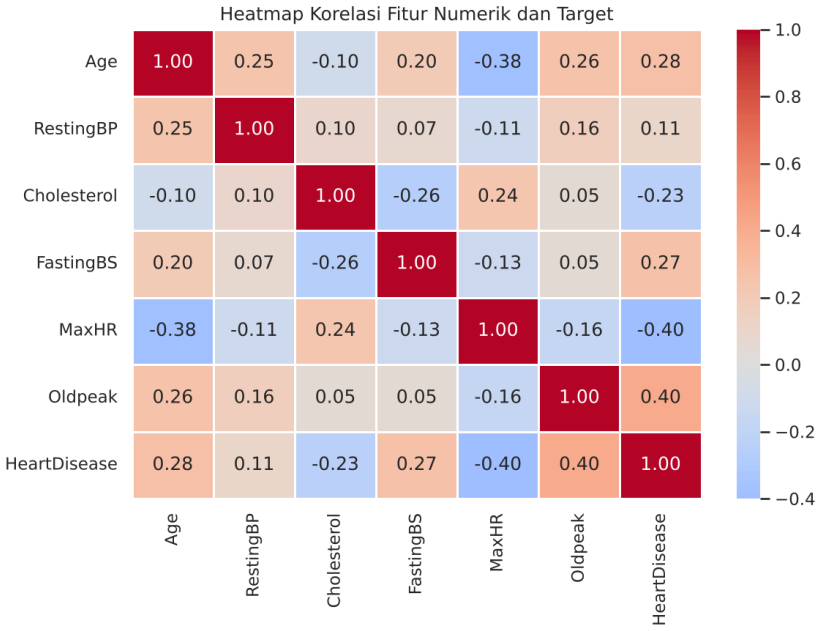
Cell 12

```
# Watermark NIM: 202331108
corr = df.select_dtypes(include='number').corr()
```

```
plt.figure(figsize=(8, 6))
sns.heatmap(corr, annot=True, fmt='.2f', cmap='coolwarm', center=0, linewidths=.5)
plt.title('Heatmap Korelasi Fitur Numerik dan Target')
simpan('06_heatmap_korelasi')
print('Korelasi terhadap HeartDisease (terurut):')
print(corr['HeartDisease'].drop('HeartDisease').sort_values(ascending=False).round(3))
```

Output:

```
Korelasi terhadap HeartDisease (terurut):
Oldpeak       0.404
Age           0.282
FastingBS     0.267
RestingBP     0.108
Cholesterol  -0.233
MaxHR        -0.400
Name: HeartDisease, dtype: float64
```



Gambar: Heatmap korelasi

Penjelasan

Heatmap korelasi memperlihatkan hubungan linear antar fitur numerik dan target. Korelasi terhadap HeartDisease: Oldpeak 0,404, Age 0,282, FastingBS 0,267, RestingBP 0,108, Cholesterol −0,233, dan MaxHR −0,400. Artinya, semakin tinggi Oldpeak dan usia, semakin besar kemungkinan sakit, sedangkan semakin tinggi MaxHR, kemungkinan sakit semakin kecil. Korelasi antar fitur tidak terlalu tinggi (terbesar Age dan MaxHR sekitar −0,38), sehingga tidak ada masalah multikolinearitas yang berarti.

Cell 13

```
# Watermark NIM: 202331108
rows = []
for c in num_cols_eda:
    q1, q3 = df[c].quantile([.25, .75]); iqr = q3 - q1
    lo, hi = q1 - 1.5*iqr, q3 + 1.5*iqr
    rows.append([c, round(lo, 2), round(hi, 2), int(((df[c] < lo) | (df[c] > hi)).sum())])
outlier = pd.DataFrame(rows, columns=['Fitur', 'Batas Bawah', 'Batas Atas', 'Jumlah Outlier'])
display(outlier)
HASIL['outlier'] = {r[0]: r[3] for r in rows}
```

Output:

	Fitur	Batas Bawah	Batas Atas	Jumlah Outlier
0	Age	27.50	79.50	0
1	RestingBP	90.00	170.00	28
2	Cholesterol	32.62	407.62	183
3	MaxHR	66.00	210.00	2
4	Oldpeak	-2.25	3.75	16

Penjelasan

Outlier dideteksi dengan aturan IQR (Q1 − 1,5×IQR dan Q3 + 1,5×IQR). Hasilnya: Age 0, RestingBP 28, Cholesterol 183, MaxHR 2, dan Oldpeak 16 outlier. Sebagian besar outlier Cholesterol adalah nilai 0 (172 dari 183), sisanya 11 nilai sangat tinggi yang masih mungkin secara klinis. Karena itu, hanya nilai nol yang ditangani, sedangkan outlier lain dipertahankan agar pasien berisiko tinggi tidak terbuang.

3.4 Data Preprocessing dan Encoding

Cell 14

```
# Watermark NIM: 202331108
# Tandai nilai tidak wajar sebagai missing (NaN)
data = df.copy()
data[['RestingBP', 'Cholesterol']] = data[['RestingBP', 'Cholesterol']].replace(0, np.nan)
print('Missing setelah penandaan nilai 0:'); print(data.isnull().sum()[lambda s: s > 0])

# Fitur (X) dan target (y)
X = data.drop(columns='HeartDisease')
y = data['HeartDisease']

# Identifikasi kolom
kolom_numerik = ['Age', 'RestingBP', 'Cholesterol', 'MaxHR', 'Oldpeak']
kolom_kategorikal = ['Sex', 'ChestPainType', 'RestingECG', 'ExerciseAngina', 'ST_Slope']
kolom_biner_numerik = ['FastingBS']   # sudah 0/1, dibiarkan apa adanya
print('\nKolom numerik     :', kolom_numerik)
print('Kolom kategorikal :', kolom_kategorikal)
print('Kolom biner (0/1) :', kolom_biner_numerik)
print('Shape X:', X.shape, '| Shape y:', y.shape)
```

Output:

```
Missing setelah penandaan nilai 0:
RestingBP       1
Cholesterol    172
dtype: int64

Kolom numerik     : ['Age', 'RestingBP', 'Cholesterol', 'MaxHR', 'Oldpeak']
Kolom kategorikal : ['Sex', 'ChestPainType', 'RestingECG', 'ExerciseAngina', 'ST_Slope']
Kolom biner (0/1) : ['FastingBS']
Shape X: (918, 11) | Shape y: (918,)
```

Penjelasan

Nilai 0 pada RestingBP dan Cholesterol diubah menjadi NaN sehingga terhitung 1 dan 172 missing value. Setelah itu data dipisah menjadi X (11 fitur) dan y (HeartDisease), dan kolom diklasifikasikan menjadi numerik (5 kolom), kategorikal (5 kolom), dan biner 0/1 (FastingBS). Imputasi belum dilakukan di sini; imputasi dilakukan di dalam pipeline agar median hanya dihitung dari data training.

Cell 15

```
# Watermark NIM: 202331108
# Train-test split 80:20, stratified
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20, random_state=RANDOM_STATE, stratify=y)
print('X_train:', X_train.shape, '| X_test:', X_test.shape)
print('\nProporsi kelas y_train (%):'); print((y_train.value_counts(normalize=True).sort_index()*100).round(2).to_string())
print('\nProporsi kelas y_test (%):'); print((y_test.value_counts(normalize=True).sort_index()*100).round(2).to_string())
HASIL['split'] = {'train': int(len(X_train)), 'test': int(len(X_test)),
                  'train_pos_pct': float(round(y_train.mean()*100, 2)), 'test_pos_pct': float(round(y_test.mean()*100, 2))}
```

Output:

```
X_train: (734, 11) | X_test: (184, 11)

Proporsi kelas y_train (%):
HeartDisease
0    44.69
1    55.31

Proporsi kelas y_test (%):
HeartDisease
```

```
0    44.57
1    55.43
```

Penjelasan

Data dibagi 80:20 dengan train_test_split(test_size=0.20, random_state=42, stratify=y), menghasilkan 734 data training dan 184 data testing. Proporsi kelas penyakit jantung terjaga: 55,31% pada training dan 55,43% pada testing, sehingga kedua subset representatif.

Cell 16

```
# Watermark NIM: 202331108
# Preprocessor Decision Tree: imputasi median + One-Hot (TANPA scaling)
pre_dt = ColumnTransformer([
    ('num', SimpleImputer(strategy='median'), kolom_numerik),
    ('cat', OneHotEncoder(drop='if_binary', handle_unknown='ignore', sparse_output=False), kolom_kategorikal),
    ('bin', 'passthrough', kolom_biner_numerik)])

# Preprocessor KNN: imputasi median + StandardScaler + One-Hot
pre_knn = ColumnTransformer([
    ('num', Pipeline([('imp', SimpleImputer(strategy='median')), ('sc', StandardScaler())]), kolom_numerik),
    ('cat', OneHotEncoder(drop='if_binary', handle_unknown='ignore', sparse_output=False), kolom_kategorikal),
    ('bin', 'passthrough', kolom_biner_numerik)])

# Demonstrasi: fit hanya pada data training
Xtr_knn = pre_knn.fit_transform(X_train)
Xte_knn = pre_knn.transform(X_test)
nama_fitur = list(pre_knn.get_feature_names_out())
print('Jumlah fitur setelah encoding:', len(nama_fitur))
print(nama_fitur)
print('\nRata-rata & std fitur numerik (TRAIN, setelah standardisasi):')
print(pd.DataFrame(Xtr_knn[:, :5], columns=kolom_numerik).agg(['mean', 'std']).round(3))
print('\nRata-rata fitur numerik (TEST, memakai statistik train):')
print(pd.DataFrame(Xte_knn[:, :5], columns=kolom_numerik).mean().round(3).to_string())
HASIL['prep'] = {'n_fitur': len(nama_fitur), 'nama_fitur': [n.split('__')[1] for n in nama_fitur],
                 'median_chol_train': float(X_train['Cholesterol'].median()), 'median_bp_train': float(X_train['RestingBP'].median())}
```

Output:

```
Jumlah fitur setelah encoding: 18
['num__Age', 'num__RestingBP', 'num__Cholesterol', 'num__MaxHR', 'num__Oldpeak', 'cat__Sex_M', 'cat__ChestPainType_ASY', 'cat__ChestPainType_ATA',
'cat__ChestPainType_NAP', 'cat__ChestPainType_TA', 'cat__RestingECG_LVH', 'cat__RestingECG_Normal', 'cat__RestingECG_ST', 'cat__ExerciseAngina_Y',
'cat__ST_Slope_Down', 'cat__ST_Slope_Flat', 'cat__ST_Slope_Up', 'bin__FastingBS']

Rata-rata & std fitur numerik (TRAIN, setelah standardisasi):
       Age  RestingBP  Cholesterol  MaxHR  Oldpeak
mean  0.000     -0.000        0.000 -0.000    0.000
std   1.001      1.001        1.001  1.001    1.001

Rata-rata fitur numerik (TEST, memakai statistik train):
Age          -0.179
RestingBP    -0.093
Cholesterol  -0.153
MaxHR         0.083
Oldpeak       0.105
```

Penjelasan

Cell ini membuat dua preprocessor. Untuk Decision Tree: imputasi median dan one-hot encoding tanpa scaling. Untuk KNN: imputasi median, standardisasi StandardScaler, dan one-hot encoding. Hasil encoding menghasilkan 18 fitur. Verifikasi anti-leakage: setelah fit_transform pada training, rata-rata fitur numerik training adalah 0 dengan simpangan baku sekitar 1 (1,001 karena pandas memakai ddof=1), sedangkan rata-rata data testing tidak tepat 0 (misalnya Age −0,179) karena ditransformasi dengan statistik training. Ini bukti bahwa data testing tidak ikut dipelajari saat scaling.

3.5 Model dan Evaluasi Decision Tree

Cell 17

```
# Watermark NIM: 202331108
# Baseline: pohon tanpa pembatasan (untuk melihat overfitting)
base_dt = Pipeline([('pre', pre_dt), ('clf', DecisionTreeClassifier(random_state=RANDOM_STATE))]).fit(X_train, y_train)
print('Baseline DT tanpa batasan -> akurasi train: %.4f | akurasi test: %.4f' % (
```

```
    base_dt.score(X_train, y_train), base_dt.score(X_test, y_test)))
print('Kedalaman pohon baseline:', base_dt.named_steps['clf'].get_depth())
HASIL['dt_baseline'] = {'train': float(base_dt.score(X_train, y_train)), 'test': float(base_dt.score(X_test, y_test)),
                        'depth': int(base_dt.named_steps['clf'].get_depth())}
```

Output:

```
Baseline DT tanpa batasan -> akurasi train: 1.0000 | akurasi test: 0.7500
Kedalaman pohon baseline: 12
```

Penjelasan

Sebagai pembanding, Decision Tree tanpa pembatasan dilatih terlebih dahulu. Pohon tumbuh hingga kedalaman 12 dan mencapai akurasi training 1,0000 tetapi akurasi testing hanya 0,7500. Selisih yang besar ini menunjukkan overfitting: pohon menghafal data training. Karena itu hiperparameter perlu dibatasi.

Cell 18

```
# Watermark NIM: 202331108
pipe_dt = Pipeline([('pre', pre_dt), ('clf', DecisionTreeClassifier(random_state=RANDOM_STATE))])
param_dt = {'clf__criterion': ['gini', 'entropy'],
            'clf__max_depth': [2, 3, 4, 5, 6, 7, 8, 10, None],
            'clf__min_samples_leaf': [1, 2, 5, 10, 20]}
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)
gs_dt = GridSearchCV(pipe_dt, param_dt, cv=cv, scoring='accuracy', n_jobs=-1).fit(X_train, y_train)
print('Parameter terbaik   :', gs_dt.best_params_)
print('Akurasi CV (train)  : %.4f' % gs_dt.best_score_)
dt_model = gs_dt.best_estimator_
HASIL['dt_params'] = {k.replace('clf__', ''): (None if v is None else (int(v) if isinstance(v, (int, np.integer)) else v))
                      for k, v in gs_dt.best_params_.items()}
HASIL['dt_cv'] = float(gs_dt.best_score_)
```

Output:

```
Parameter terbaik   : {'clf__criterion': 'gini', 'clf__max_depth': 4, 'clf__min_samples_leaf': 10}
Akurasi CV (train)  : 0.8406
```

Penjelasan

GridSearchCV mencoba 90 kombinasi (2 criterion × 9 max_depth × 5 min_samples_leaf) dengan 5-fold stratified cross-validation pada data training saja. Kombinasi terbaik: criterion='gini', max_depth=4, min_samples_leaf=10, dengan akurasi CV 0,8406. Pembatasan kedalaman dan ukuran daun membuat pohon lebih sederhana dan lebih mampu menggeneralisasi.

Cell 19

```
# Watermark NIM: 202331108
y_pred_dt = dt_model.predict(X_test)
y_prob_dt = dt_model.predict_proba(X_test)[:, 1]
acc_dt = accuracy_score(y_test, y_pred_dt); pre_dt_s = precision_score(y_test, y_pred_dt)
rec_dt = recall_score(y_test, y_pred_dt); f1_dt = f1_score(y_test, y_pred_dt); auc_dt = roc_auc_score(y_test, y_prob_dt)
print(f'Accuracy  : {acc_dt:.4f}\nPrecision : {pre_dt_s:.4f}\nRecall    : {rec_dt:.4f}\nF1-score  : {f1_dt:.4f}\nROC-AUC   : {auc_dt:.4f}')
print('\nClassification Report:')
print(classification_report(y_test, y_pred_dt, target_names=['Normal (0)', 'Penyakit Jantung (1)'], digits=4))
cm_dt = confusion_matrix(y_test, y_pred_dt)
tn, fp, fn, tp = cm_dt.ravel()
print(f'TN={tn}  FP={fp}  FN={fn}  TP={tp}')
HASIL['dt'] = {'accuracy': acc_dt, 'precision': pre_dt_s, 'recall': rec_dt, 'f1': f1_dt, 'auc': auc_dt,
               'tn': int(tn), 'fp': int(fp), 'fn': int(fn), 'tp': int(tp),
               'train_acc': float(dt_model.score(X_train, y_train)), 'depth': int(dt_model.named_steps['clf'].get_depth()),
               'leaves': int(dt_model.named_steps['clf'].get_n_leaves())}

fig, ax = plt.subplots(figsize=(5.5, 4.5))
ConfusionMatrixDisplay(cm_dt, display_labels=['Normal', 'Jantung']).plot(ax=ax, cmap='Blues', colorbar=False)
ax.set_title('Confusion Matrix - Decision Tree'); ax.set_xlabel('Label Prediksi'); ax.set_ylabel('Label Aktual')
simpan('07_cm_decision_tree')
```

Output:

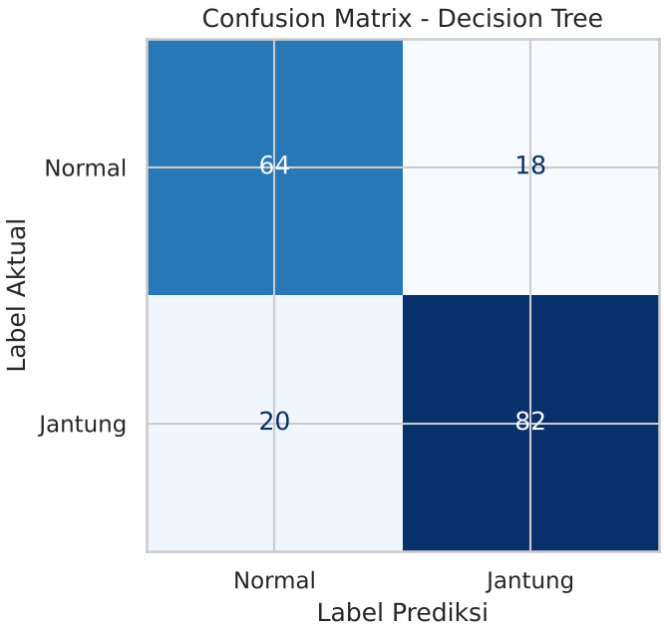
```
Accuracy  : 0.7935
Precision : 0.8200
Recall    : 0.8039
F1-score  : 0.8119
ROC-AUC   : 0.8544
```

```
Classification Report:
                    precision    recall  f1-score   support

        Normal (0)     0.7619    0.7805    0.7711        82
Penyakit Jantung (1)     0.8200    0.8039    0.8119       102

          accuracy                         0.7935       184
         macro avg     0.7910    0.7922    0.7915       184
      weighted avg     0.7941    0.7935    0.7937       184

TN=64  FP=18  FN=20  TP=82
```



Gambar: Confusion matrix Decision Tree

Penjelasan

Model terbaik diuji pada 184 data testing. Hasil Decision Tree: Accuracy 0,7935, Precision 0,8200, Recall 0,8039, F1-score 0,8119, dan ROC-AUC 0,8544. Confusion matrix menunjukkan TN = 64, FP = 18, FN = 20, TP = 82. Artinya 82 dari 102 pasien sakit terdeteksi, tetapi 20 pasien sakit terlewat (diprediksi normal) dan 18 pasien normal salah diprediksi sakit. Akurasi training 0,8624 vs testing 0,7935: kesenjangan ±0,07 jauh lebih kecil dibanding pohon tanpa batasan (±0,25).

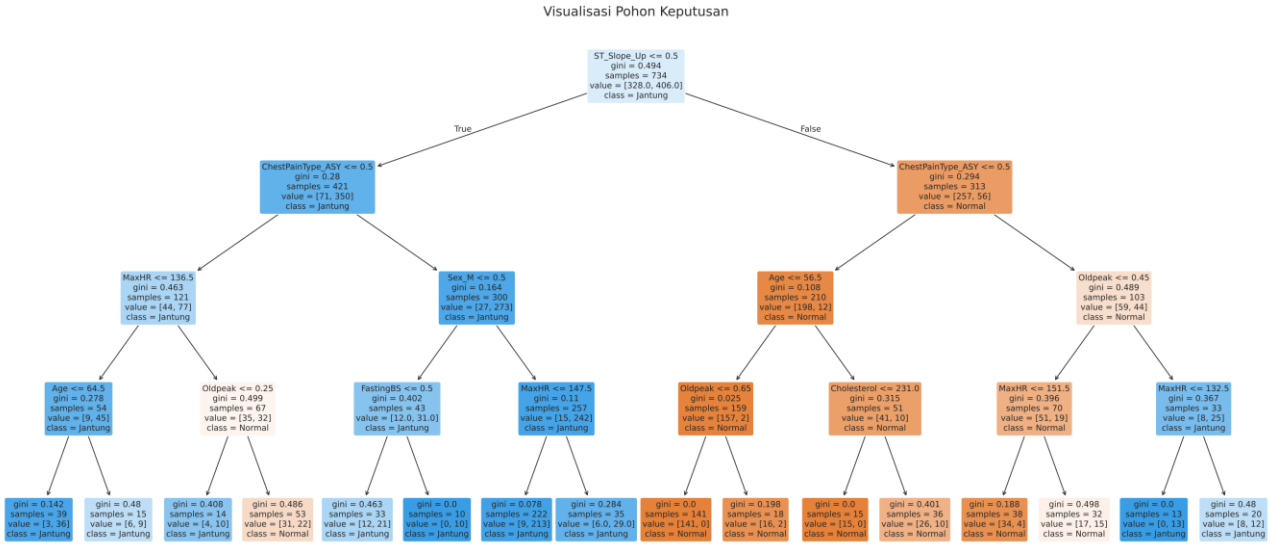
Cell 20

```
# Watermark NIM: 20231108
clf_dt = dt_model.named_steps['clf']
fitur_dt = [n.split('__')[1] for n in dt_model.named_steps['pre'].get_feature_names_out()]
depth_plot = None if clf_dt.get_depth() <= 4 else 3
plt.figure(figsize=(22, 10))
plot_tree(clf_dt, feature_names=fitur_dt, class_names=['Normal', 'Jantung'], filled=True, rounded=True,
          fontsize=10, max_depth=depth_plot, proportion=False)
plt.title('Visualisasi Pohon Keputusan' + (' (ditampilkan sampai kedalaman 3)' if depth_plot else ''), fontsize=16)
simpan('08_plot_tree')
print(export_text(clf_dt, feature_names=fitur_dt, max_depth=3))
```

Output:

```
|--- ST_Slope_Up <= 0.50
|   |--- ChestPainType_ASY <= 0.50
|   |   |--- MaxHR <= 136.50
|   |   |   |--- Age <= 64.50
|   |   |   |   |--- class: 1
|   |   |   |--- Age >  64.50
|   |   |   |   |--- class: 1
|   |   |--- MaxHR >  136.50
|   |   |   |--- Oldpeak <= 0.25
|   |   |   |   |--- class: 1
|   |   |   |--- Oldpeak >  0.25
|   |   |   |   |--- class: 0
```

```
|   |--- ChestPainType_ASY >  0.50
|   |   |--- Sex_M <= 0.50
|   |   |   |--- FastingBS <= 0.50
|   |   |   |   |--- class: 1
|   |   |   |--- FastingBS >  0.50
|   |   |   |   |--- class: 1
|   |   |--- Sex_M >  0.50
|   |   |   |--- MaxHR <= 147.50
|   |   |   |   |--- class: 1
|   |   |   |--- MaxHR >  147.50
|   |   |   |   |--- class: 1
|--- ST_Slope_Up >  0.50
|   |--- ChestPainType_ASY <= 0.50
|   |   |--- Age <= 56.50
|   |   |   |--- Oldpeak <= 0.65
|   |   |   |   |--- class: 0
|   |   |   |--- Oldpeak >  0.65
|   |   |   |   |--- class: 0
|   |   |--- Age >  56.50
|   |   |   |--- Cholesterol <= 231.00
|   |   |   |   |--- class: 0
|   |   |   |--- Cholesterol >  231.00
|   |   |   |   |--- class: 0
|   |--- ChestPainType_ASY >  0.50
|   |   |--- Oldpeak <= 0.45
|   |   |   |--- MaxHR <= 151.50
|   |   |   |   |--- class: 0
|   |   |   |--- MaxHR >  151.50
|   |   |   |   |--- class: 0
|   |   |--- Oldpeak >  0.45
|   |   |   |--- MaxHR <= 132.50
|   |   |   |   |--- class: 1
|   |   |   |--- MaxHR >  132.50
|   |   |   |   |--- class: 1
```



Gambar: Visualisasi pohon keputusan (plot_tree)

Penjelasan

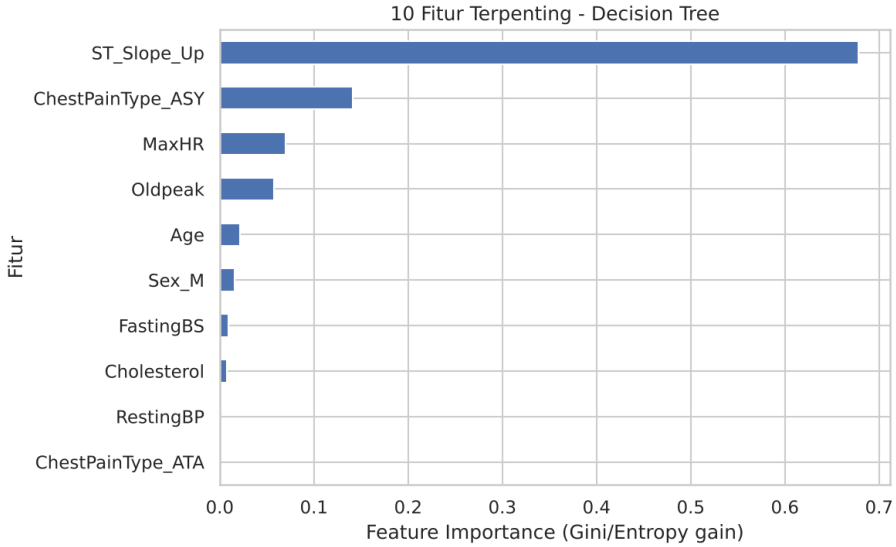
plot_tree memvisualisasikan pohon keputusan (kedalaman 4, 16 daun). Simpul akar menguji ST_Slope_Up <= 0.5. Jika kondisi benar (kemiringan ST bukan Up), 350 dari 421 pasien adalah penyakit jantung, sedangkan jika salah (ST_Slope = Up) 257 dari 313 pasien normal. Pada tiap cabang, pohon melanjutkan dengan menguji ChestPainType_ASY, MaxHR, Oldpeak, Age, dan lainnya sampai mencapai daun. Warna biru menandakan mayoritas penyakit jantung dan oranye mayoritas normal; semakin pekat, semakin murni simpul. Dengan demikian, Decision Tree mengambil keputusan lewat rangkaian aturan jika-maka, dan export_text menampilkan aturan ini dalam bentuk teks.

Cell 21

```
# Watermark NIM: 20231108
imp = pd.Series(clf_dt.feature_importances_, index=fitur_dt).sort_values(ascending=False)
print(imp.round(4).head(10))
HASIL['dt_importance'] = {k: float(v) for k, v in imp.head(6).items()}
plt.figure(figsize=(8, 5))
imp.head(10).sort_values().plot(kind='barh', color='#4C72B0')
plt.title('10 Fitur Terpenting - Decision Tree'); plt.xlabel('Feature Importance (Gini/Entropy gain)'); plt.ylabel('Fitur')
simpan('09_importance_dt')
```

Output:

```
ST_Slope_Up         0.6778
ChestPainType_ASY    0.1412
MaxHR                0.0697
Oldpeak              0.0576
Age                  0.0216
Sex_M                0.0159
FastingBS            0.0090
Cholesterol          0.0072
RestingBP            0.0000
ChestPainType_ATA    0.0000
dtype: float64
```



Gambar: Feature importance Decision Tree

Penjelasan

Feature importance menunjukkan kontribusi tiap fitur terhadap penurunan impurity. ST_Slope_Up paling dominan (0,6778), diikuti ChestPainType_ASY (0,1412), MaxHR (0,0697), dan Oldpeak (0,0576), sedangkan RestingBP bernilai 0 karena tidak dipakai pohon. Urutan ini konsisten dengan EDA yang menunjukkan ST_Slope dan ChestPainType sebagai fitur paling berhubungan dengan target.

3.6 Model dan Evaluasi KNN

Cell 22

```
# Watermark NIM: 202331108
hasil_k = []
for k in range(1, 31):
    pipe_k = Pipeline([('pre', pre_knn), ('clf', KNeighborsClassifier(n_neighbors=k))])
    cv_acc = cross_val_score(pipe_k, X_train, y_train, cv=cv, scoring='accuracy').mean()
    pipe_k.fit(X_train, y_train)
    hasil_k.append([k, cv_acc, pipe_k.score(X_train, y_train), pipe_k.score(X_test, y_test)])
tabel_k = pd.DataFrame(hasil_k, columns=['K', 'Akurasi CV (train)', 'Akurasi Train', 'Akurasi Test'])
display(tabel_k.round(4).set_index('K').T)

k_terbaik = int(tabel_k.loc[tabel_k['Akurasi CV (train)'].idxmax(), 'K'])
print('K terbaik (akurasi CV tertinggi):', k_terbaik, '| akurasi CV = %.4f' % tabel_k['Akurasi CV (train)'].max())
HASIL['knn_k'] = {'k_terbaik': k_terbaik, 'cv_terbaik': float(tabel_k['Akurasi CV (train)'].max()),
```

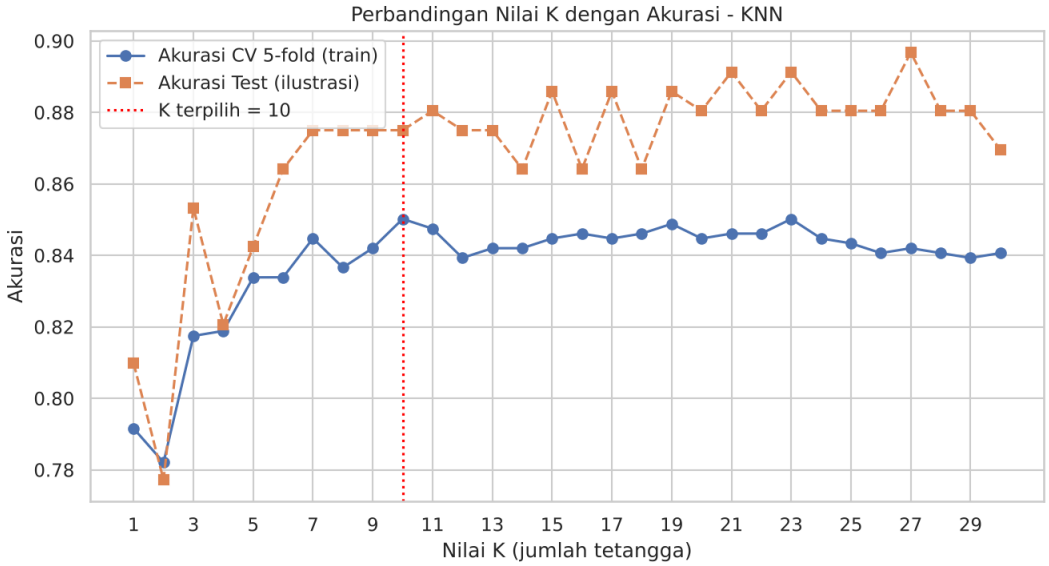
```
            'tabel': tabel_k.round(4).to_dict(orient='list')}

plt.figure(figsize=(9, 5))
plt.plot(tabel_k['K'], tabel_k['Akurasi CV (train)'], marker='o', label='Akurasi CV 5-fold (train)')
plt.plot(tabel_k['K'], tabel_k['Akurasi Test'], marker='s', linestyle='--', label='Akurasi Test (ilustrasi)')
plt.axvline(k_terbaik, color='red', linestyle=':', label=f'K terpilih = {k_terbaik}')
plt.title('Perbandingan Nilai K dengan Akurasi - KNN'); plt.xlabel('Nilai K (jumlah tetangga)'); plt.ylabel('Akurasi')
plt.xticks(range(1, 31, 2)); plt.legend()
simpan('10_k_vs_akurasi')
```

Output: tabel akurasi tiap nilai K.

K	Akurasi CV (train)	Akurasi Train	Akurasi Test	K	Akurasi CV (train)	Akurasi Train	Akurasi Test
1	0.7916	1.0000	0.8098	16	0.8461	0.8569	0.8641
2	0.7820	0.8951	0.7772	17	0.8447	0.8638	0.8859
3	0.8175	0.9046	0.8533	18	0.8461	0.8542	0.8641
4	0.8189	0.8828	0.8207	19	0.8488	0.8569	0.8859
5	0.8338	0.8719	0.8424	20	0.8447	0.8569	0.8804
6	0.8338	0.8678	0.8641	21	0.8461	0.8610	0.8913
7	0.8447	0.8678	0.8750	22	0.8461	0.8610	0.8804
8	0.8366	0.8692	0.8750	23	0.8502	0.8569	0.8913
9	0.8420	0.8665	0.8750	24	0.8447	0.8597	0.8804
10	0.8502	0.8678	0.8750	25	0.8434	0.8529	0.8804
11	0.8475	0.8638	0.8804	26	0.8406	0.8542	0.8804
12	0.8393	0.8610	0.8750	27	0.8420	0.8529	0.8967
13	0.8420	0.8542	0.8750	28	0.8406	0.8529	0.8804
14	0.8420	0.8569	0.8641	29	0.8393	0.8515	0.8804
15	0.8447	0.8624	0.8859	30	0.8406	0.8488	0.8696

K terbaik (akurasi CV tertinggi): 10 | akurasi CV = 0.8502



Gambar: Grafik nilai K terhadap akurasi KNN

Penjelasan

Nilai K diuji dari 1 sampai 30 memakai pipeline (imputasi, standardisasi, KNN) dengan 5-fold CV pada data training. Pada K = 1 akurasi training 1,0000 tetapi akurasi CV hanya 0,7916 (overfitting). Akurasi CV naik seiring K bertambah dan mencapai puncak 0,8502 pada **K = 10** (K = 23 memiliki akurasi CV yang sama; dipilih K lebih kecil karena menghasilkan model lebih sederhana dan prediksi lebih cepat). Setelah K di atas 10 kurva cenderung datar sehingga tidak ada peningkatan berarti. Kurva akurasi testing digambar hanya sebagai ilustrasi dan tidak dipakai untuk memilih K, sehingga data testing tetap bersih.

Cell 23

```
# Watermark NIM: 202331108
knn_model = Pipeline([('pre', pre_knn), ('clf', KNeighborsClassifier(n_neighbors=k_terbaik))]).fit(X_train, y_train)
y_pred_knn = knn_model.predict(X_test)
y_prob_knn = knn_model.predict_proba(X_test)[:, 1]
acc_knn = accuracy_score(y_test, y_pred_knn); pre_knn_s = precision_score(y_test, y_pred_knn)
rec_knn = recall_score(y_test, y_pred_knn); f1_knn = f1_score(y_test, y_pred_knn); auc_knn = roc_auc_score(y_test, y_prob_knn)
print(f'K = {k_terbaik}')
print(f'Accuracy  : {acc_knn:.4f}\nPrecision : {pre_knn_s:.4f}\nRecall    : {rec_knn:.4f}\nF1-score  : {f1_knn:.4f}\nROC-AUC   : {auc_knn:.4f}')
print('\nClassification Report:')
print(classification_report(y_test, y_pred_knn, target_names=['Normal (0)', 'Penyakit Jantung (1)'], digits=4))
cm_knn = confusion_matrix(y_test, y_pred_knn)
tn, fp, fn, tp = cm_knn.ravel()
print(f'TN={tn}  FP={fp}  FN={fn}  TP={tp}')
HASIL['knn'] = {'k': k_terbaik, 'accuracy': acc_knn, 'precision': pre_knn_s, 'recall': rec_knn, 'f1': f1_knn, 'auc': auc_knn,
                'tn': int(tn), 'fp': int(fp), 'fn': int(fn), 'tp': int(tp), 'train_acc': float(knn_model.score(X_train, y_train))}

fig, ax = plt.subplots(figsize=(5.5, 4.5))
ConfusionMatrixDisplay(cm_knn, display_labels=['Normal', 'Jantung']).plot(ax=ax, cmap='Greens', colorbar=False)
ax.set_title(f'Confusion Matrix - KNN (K = {k_terbaik})'); ax.set_xlabel('Label Prediksi'); ax.set_ylabel('Label Aktual')
simpan('11_cm_knn')
```

Output:

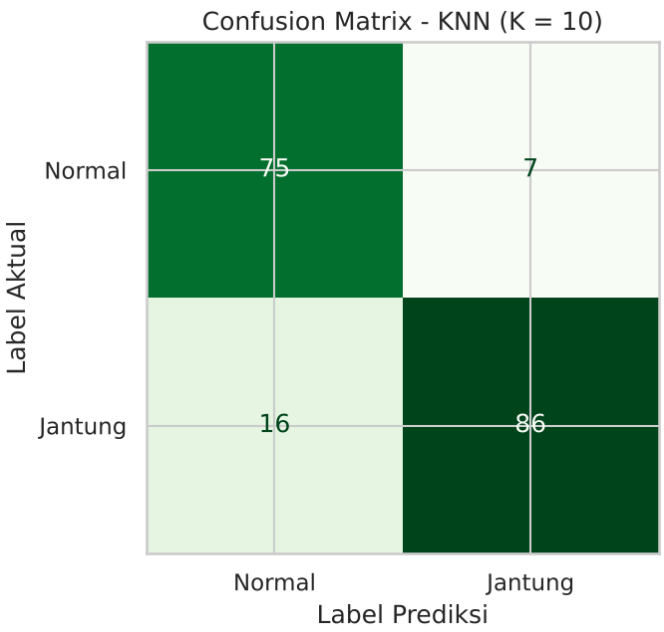
```
K = 10
Accuracy  : 0.8750
Precision : 0.9247
Recall    : 0.8431
F1-score  : 0.8821
ROC-AUC   : 0.9356

Classification Report:
                   precision    recall  f1-score   support

       Normal (0)     0.8242    0.9146    0.8671        82
Penyakit Jantung (1)     0.9247    0.8431    0.8821       102

         accuracy                         0.8750       184
        macro avg     0.8745    0.8789    0.8746       184
     weighted avg     0.8799    0.8750    0.8754       184

TN=75  FP=7  FN=16  TP=86
```



Gambar: Confusion matrix KNN

Penjelasan

KNN dengan K = 10 dievaluasi pada data testing: Accuracy 0,8750, Precision 0,9247, Recall 0,8431, F1-score 0,8821, dan ROC-AUC 0,9356. Confusion matrix: TN = 75, FP = 7, FN = 16, TP = 86. Hanya 7 pasien normal yang salah diprediksi sakit, tetapi 16 pasien sakit masih terlewat. Cara

kerjanya: untuk tiap pasien baru, KNN menghitung jarak Euclidean ke seluruh pasien training pada ruang 18 fitur terstandardisasi, mengambil 10 tetangga terdekat, dan memilih kelas mayoritas.

3.7 Perbandingan Decision Tree dan KNN

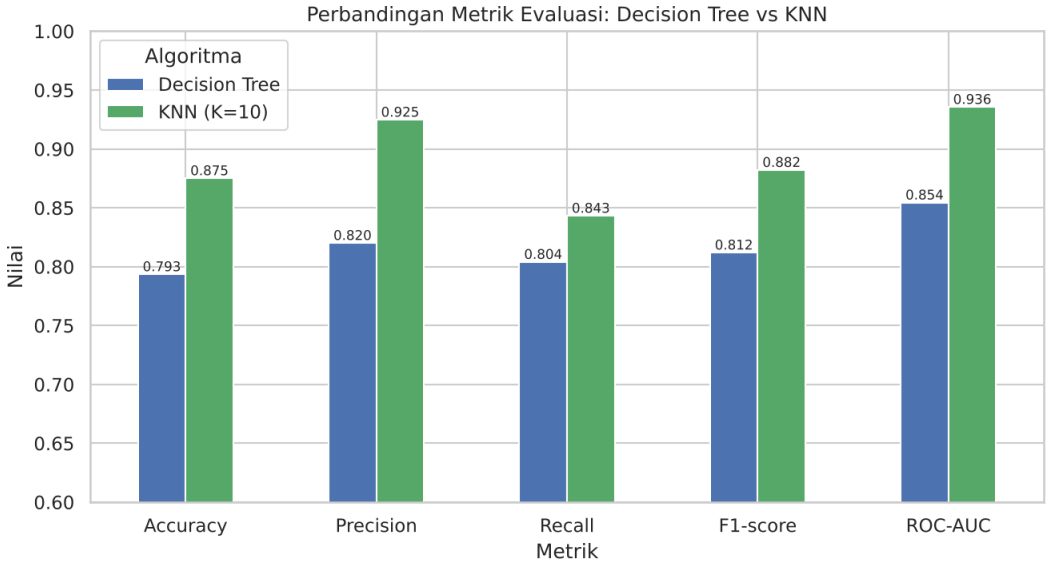
Cell 24

```
# Watermark NIM: 202331108
perb = pd.DataFrame({
    'Algoritma': ['Decision Tree', f'KNN (K={k_terbaik})'],
    'Accuracy': [acc_dt, acc_knn], 'Precision': [pre_dt_s, pre_knn_s],
    'Recall': [rec_dt, rec_knn], 'F1-score': [f1_dt, f1_knn], 'ROC-AUC': [auc_dt, auc_knn]}).set_index('Algoritma')
display(perb.round(4))

ax = perb[['Accuracy', 'Precision', 'Recall', 'F1-score', 'ROC-AUC']].T.plot(kind='bar', figsize=(9, 5), color=['#4C72B0', '#55A868'], rot=0)
ax.set_title('Perbandingan Metrik Evaluasi: Decision Tree vs KNN'); ax.set_xlabel('Metrik'); ax.set_ylabel('Nilai')
ax.set_ylim(0.6, 1.0); ax.legend(title='Algoritma')
for c in ax.containers: ax.bar_label(c, fmt='%.3f', fontsize=8)
simpan('12_perbandingan_metrik')
```

Output:

Algoritma	Accuracy	Precision	Recall	F1-score	ROC-AUC
Decision Tree	0.7935	0.8200	0.8039	0.8119	0.8544
KNN (K=10)	0.8750	0.9247	0.8431	0.8821	0.9356



Gambar: Perbandingan metrik Decision Tree dan KNN

Penjelasan

Tabel dan grafik batang membandingkan lima metrik kedua model pada data testing yang sama. KNN lebih tinggi pada seluruh metrik: Accuracy 0,8750 vs 0,7935, Precision 0,9247 vs 0,8200, Recall 0,8431 vs 0,8039, F1-score 0,8821 vs 0,8119, dan ROC-AUC 0,9356 vs 0,8544.

Tabel perbandingan Decision Tree dan KNN (data uji 184 pasien; kelas positif = penyakit jantung).

Algoritma	Accuracy	Precision	Recall	F1-score	Kelebihan	Kekurangan
Decision Tree	0.7935	0.8200	0.8039	0.8119	Mudah diinterpretasi (aturan jika-maka), tidak perlu scaling, menangani fitur campuran, memberi feature importance	Rentan overfitting, tidak stabil terhadap perubahan data kecil, perlu pembatasan kedalaman
KNN (K=10)	0.8750	0.9247	0.8431	0.8821	Sederhana, tanpa training eksplisit, mampu menangkap batas keputusan non-linear, efektif bila fitur sudah diskalakan	Sensitif terhadap skala dan fitur tidak relevan, prediksi lambat pada data besar, hasil bergantung pada K dan jarak

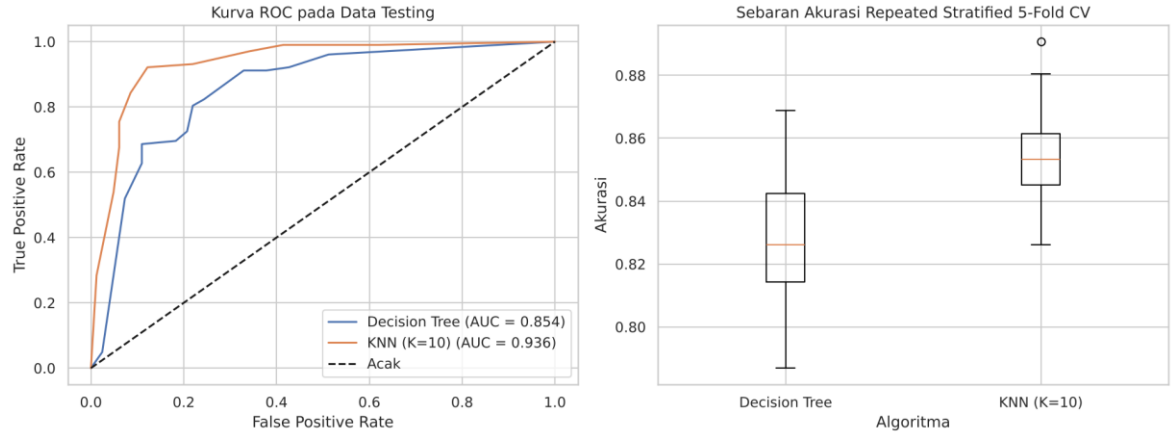
Cell 25

```
# Watermark NIM: 202331108
fig, ax = plt.subplots(1, 2, figsize=(13, 5))
for nama, prob, auc in [('Decision Tree', y_prob_dt, auc_dt), (f'KNN (K={k_terbaik})', y_prob_knn, auc_knn)]:
    fpr, tpr, _ = roc_curve(y_test, prob); ax[0].plot(fpr, tpr, label=f'{nama} (AUC = {auc:.3f})')
ax[0].plot([0, 1], [0, 1], 'k--', label='Acak'); ax[0].set_title('Kurva ROC pada Data Testing')
ax[0].set_xlabel('False Positive Rate'); ax[0].set_ylabel('True Positive Rate'); ax[0].legend()

# Uji ketahanan: Repeated Stratified 5-Fold (3x) pada seluruh data memakai hiperparameter terpilih
rcv = RepeatedStratifiedKFold(n_splits=5, n_repeats=3, random_state=RANDOM_STATE)
dt_cv = cross_val_score(Pipeline([('pre', pre_dt), ('clf', DecisionTreeClassifier(random_state=RANDOM_STATE, **HASIL['dt_params']))]), X, y, cv=rcv)
knn_cv = cross_val_score(Pipeline([('pre', pre_knn), ('clf', KNeighborsClassifier(n_neighbors=k_terbaik))]), X, y, cv=rcv)
print(f'Repeated CV (15 lipatan) Decision Tree : {dt_cv.mean():.4f} +/- {dt_cv.std():.4f}')
print(f'Repeated CV (15 lipatan) KNN           : {knn_cv.mean():.4f} +/- {knn_cv.std():.4f}')
HASIL['rcv'] = {'dt_mean': float(dt_cv.mean()), 'dt_std': float(dt_cv.std()), 'knn_mean': float(knn_cv.mean()), 'knn_std': float(knn_cv.std())}
ax[1].boxplot([dt_cv, knn_cv])
ax[1].set_xticklabels(['Decision Tree', f'KNN (K={k_terbaik})'])
ax[1].set_title('Sebaran Akurasi Repeated Stratified 5-Fold CV'); ax[1].set_xlabel('Algoritma'); ax[1].set_ylabel('Akurasi')
simpan('13_roc_dan_rcv')
```

Output:

```
Repeated CV (15 lipatan) Decision Tree : 0.8286 +/- 0.0240
Repeated CV (15 lipatan) KNN           : 0.8551 +/- 0.0160
```



Gambar: Kurva ROC dan sebaran akurasi Repeated CV

Penjelasan

Kurva ROC KNN berada di atas kurva Decision Tree (AUC 0,9356 vs 0,8544), menandakan KNN lebih baik memisahkan kedua kelas di seluruh ambang probabilitas. Karena data uji hanya 184 pasien, dilakukan uji ketahanan Repeated Stratified 5-Fold CV (3 ulangan, 15 lipatan) pada seluruh data dengan hiperparameter terpilih: Decision Tree 0,8286 ± 0,0240 dan KNN 0,8551 ± 0,0160. Arah hasilnya sama dengan data uji, dan KNN juga lebih stabil (simpangan baku lebih kecil). Catatan: hiperparameter dipilih dari data training, sehingga angka ini bersifat pelengkap, bukan pengganti evaluasi data uji.

Cell 26

```
# Watermark NIM: 202331108
kelebihan_kekurangan = pd.DataFrame({
    'Algoritma': ['Decision Tree', 'KNN'],
    'Kelebihan': ['Mudah diinterpretasi (aturan if-then), tidak perlu scaling, menangani fitur campuran, memberi feature importance',
                  'Sederhana, tanpa proses training eksplisit, mampu menangkap batas keputusan non-linear, efektif bila fitur sudah diskalakan'],
    'Kekurangan': ['Rentan overfitting, tidak stabil terhadap perubahan data kecil, perlu pembatasan kedalaman',
                   'Sensitif terhadap skala dan fitur tidak relevan, prediksi lambat pada data besar, hasil bergantung pada K dan jarak']})
pd.set_option('display.max_colwidth', None)
display(kelebihan_kekurangan)

lebih = lambda a, b: 'Decision Tree' if a > b else ('KNN' if b > a else 'Sama')
print('Akurasi lebih tinggi   :', lebih(acc_dt, acc_knn))
print('Precision lebih baik   :', lebih(pre_dt_s, pre_knn_s))
print('Recall lebih baik      :', lebih(rec_dt, rec_knn))
print('F1-score lebih baik    :', lebih(f1_dt, f1_knn))
```

```
print('ROC-AUC lebih baik     :', lebih(auc_dt, auc_knn))
HASIL['pemenang'] = {'accuracy': lebih(acc_dt, acc_knn), 'precision': lebih(pre_dt_s, pre_knn_s), 'recall': lebih(rec_dt, rec_knn),
                     'f1': lebih(f1_dt, f1_knn), 'auc': lebih(auc_dt, auc_knn)}
with open('hasil.json', 'w') as f: json.dump(HASIL, f, indent=1, default=str)
print('\nSeluruh hasil disimpan ke hasil.json')
```

Output:

```
Akurasi lebih tinggi   : KNN
Precision lebih baik   : KNN
Recall lebih baik      : KNN
F1-score lebih baik    : KNN
ROC-AUC lebih baik     : KNN

Seluruh hasil disimpan ke hasil.json
```

Penjelasan

Cell terakhir merangkum kelebihan dan kekurangan masing-masing algoritma serta menentukan pemenang tiap metrik secara otomatis dari angka hasil eksekusi: KNN unggul pada Accuracy, Precision, Recall, F1-score, dan ROC-AUC. Seluruh hasil disimpan ke hasil.json agar angka pada laporan ini dapat ditelusuri ke eksekusi kode.

Analisis Perbandingan

1. **Akurasi lebih tinggi:** KNN (0,8750) lebih tinggi daripada Decision Tree (0,7935), selisih sekitar 8,15 poin persen (setara 15 pasien dari 184 data uji).

2. **Precision lebih baik:** KNN (0,9247) dibanding Decision Tree (0,8200). KNN hanya 7 kali salah menandai pasien normal sebagai sakit, sedangkan Decision Tree 18 kali.

3. **Recall lebih baik:** KNN (0,8431) sedikit lebih baik daripada Decision Tree (0,8039). Keduanya masih melewatkan pasien sakit (FN KNN = 16, Decision Tree = 20), sehingga pada penggunaan klinis recall masih perlu ditingkatkan.

4. **F1-score:** KNN (0,8821) lebih tinggi daripada Decision Tree (0,8119), konsisten dengan keunggulan precision dan recall.

5. **Mengapa hasilnya berbeda:** Decision Tree membuat batas keputusan berbentuk persegi panjang (sumbu sejajar) berdasarkan satu fitur per pemisahan; pohon yang dibatasi sampai kedalaman 4 hanya memakai sebagian fitur (RestingBP tidak dipakai) dan rentan berubah jika data berubah sedikit. KNN memakai jarak pada seluruh 18 fitur terstandardisasi secara bersamaan sehingga mampu menangkap kombinasi fitur dan batas keputusan yang lebih fleksibel. Preprocessing yang tepat (imputasi median dan standardisasi) juga sangat membantu KNN. Pola ini sejalan dengan Qumsiyeh et al. (2026) pada dataset yang sama (KNN lebih tinggi daripada Decision Tree), sedangkan hasil Budi (2026) dan Pavithraa dan Sivaprasad (2022) berbeda karena dataset dan skema eksperimennya berbeda.

6. **Algoritma yang lebih sesuai:** Berdasarkan hasil eksperimen, KNN lebih sesuai untuk dataset ini karena unggul pada seluruh metrik, baik pada data uji maupun pada uji ketahanan Repeated CV. Namun Decision Tree tetap bermanfaat bila interpretabilitas aturan keputusan menjadi prioritas.

BAB IV

PENUTUP

Berdasarkan praktikum prediksi penyakit jantung menggunakan dataset 01_heart_disease.csv, diperoleh kesimpulan berikut yang seluruhnya bersumber dari hasil eksekusi program.

1. **Hasil EDA dan preprocessing.** Dataset terdiri dari 918 pasien dan 12 kolom tanpa NaN dan tanpa duplikat. Target relatif seimbang (410 normal atau 44,66% dan 508 penyakit jantung atau 55,34%). EDA menemukan 172 nilai Cholesterol = 0 dan 1 nilai RestingBP = 0 yang tidak wajar, sehingga diubah menjadi NaN dan diimputasi median di dalam pipeline. Fitur yang paling berhubungan dengan penyakit jantung adalah ST_Slope, ExerciseAngina, ChestPainType (ASY), Oldpeak, dan MaxHR. Data dibagi 80:20 (734 training dan 184 testing) secara stratified dengan random_state=42, dan seluruh preprocessing di-fit hanya pada data training.

2. **Hasil Decision Tree.** Pohon tanpa batasan mengalami overfitting (akurasi training 1,0000 vs testing 0,7500). Setelah tuning (gini, max_depth=4, min_samples_leaf=10), diperoleh Accuracy 0,7935, Precision 0,8200, Recall 0,8039, dan F1-score 0,8119. Fitur paling berpengaruh adalah ST_Slope_Up (0,6778) dan ChestPainType_ASY (0,1412).

3. **Hasil KNN.** Dengan standardisasi dan pemilihan K melalui cross-validation, diperoleh K = 10. KNN menghasilkan Accuracy 0,8750, Precision 0,9247, Recall 0,8431, dan F1-score 0,8821.

4. **Perbandingan akurasi.** KNN (0,8750) lebih akurat daripada Decision Tree (0,7935), dan juga lebih tinggi pada precision, recall, F1-score, dan ROC-AUC (0,9356 vs 0,8544). Uji Repeated Stratified 5-Fold CV menunjukkan arah yang sama (KNN 0,8551 ± 0,0160 vs Decision Tree 0,8286 ± 0,0240).

5. **Algoritma terbaik.** Berdasarkan hasil eksperimen, KNN adalah algoritma dengan performa terbaik pada dataset ini, sedangkan Decision Tree unggul dalam interpretabilitas karena menghasilkan aturan jika-maka yang mudah dibaca.

6. **Keterbatasan.** Hasil berasal dari satu dataset dan satu pembagian data uji berukuran 184 pasien, sehingga selisih performa perlu dibaca dengan hati-hati. Pada konteks medis, kedua model masih melewatkan pasien sakit (FN: Decision Tree 20, KNN 16), sehingga penelitian lanjutan dapat menguji model lain, tuning hiperparameter yang lebih luas, dan validasi pada data eksternal. Model ini hanya untuk tujuan pembelajaran dan bukan alat diagnosis.

DAFTAR PUSTAKA

Budi, B. (2026). Heart Disease Prediction Using KNN, Decision Tree, and Naïve Bayes. Jurnal Andalas: Rekayasa dan Penerapan Teknologi (JARPeT), 6(1), 40–46. https://doi.org/10.25077/jarpet.v6i1.142

Canbek, G., Taskaya Temizel, T., & Sagiroglu, S. (2022). PToPI: A Comprehensive Review, Analysis, and Knowledge Representation of Binary Classification Performance Measures/Metrics. SN Computer Science, 4(1), 13. https://doi.org/10.1007/s42979-022-01409-1

Kapoor, S., & Narayanan, A. (2023). Leakage and the reproducibility crisis in machine-learning-based science. Patterns, 4(9), 100804. https://doi.org/10.1016/j.patter.2023.100804

Pavithraa, G., & Sivaprasad. (2022). Analysis and comparison of prediction of heart disease using Novel K nearest neighbor and decision tree algorithm. Cardiometry, 25, 773–777. https://doi.org/10.18137/cardiometry.2022.25.773777

Pratama, Y. D., & Salam, A. (2025). Comparison of Data Normalization Techniques on KNN Classification Performance for Pima Indians Diabetes Dataset. Journal of Applied Informatics and Computing, 9(3), 693–706. https://doi.org/10.30871/jaic.v9i3.9353

Qumsiyeh, E., Al-Wirdian, Q., & Ersoz, N. S. (2026). Leveraging Clinical Data for Early Heart Disease Prediction: A Machine Learning Approach With Interpretability Analysis. Biomedical Engineering and Computational Biology, 17. https://doi.org/10.1177/11795972261446822

Sarker, I. H. (2021). Machine Learning: Algorithms, Real-World Applications and Research Directions. SN Computer Science, 2, 160. https://doi.org/10.1007/s42979-021-00592-x

Tharwat, A. (2021). Classification assessment methods. Applied Computing and Informatics, 17(1), 168–192. https://doi.org/10.1016/j.aci.2018.08.003